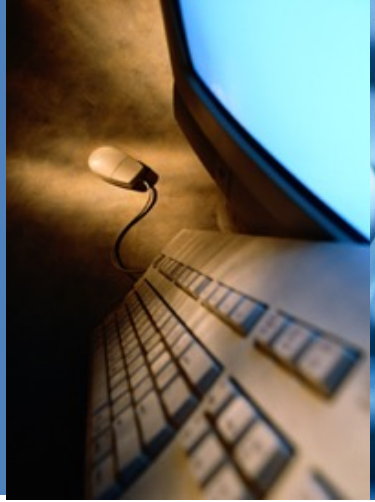
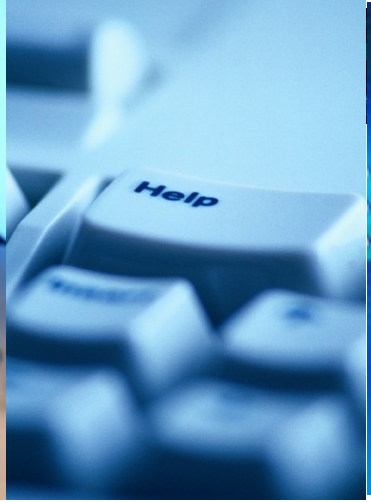





分布式系统
Distributed Systems

陈鹏飞
计算机学院
chenpf7@mail.sysu.edu.cn
办公室：学院楼310
主页：http://sdcs.sysu.edu.cn/node/3747

第九讲 — 分布式系统共识



Slides come from drdr.xp@gmail.com, Amir H. Payberah, Jinyang Li





背景

- 分布式系统协作

  多个部分协同工作，对外表现出一个整体；

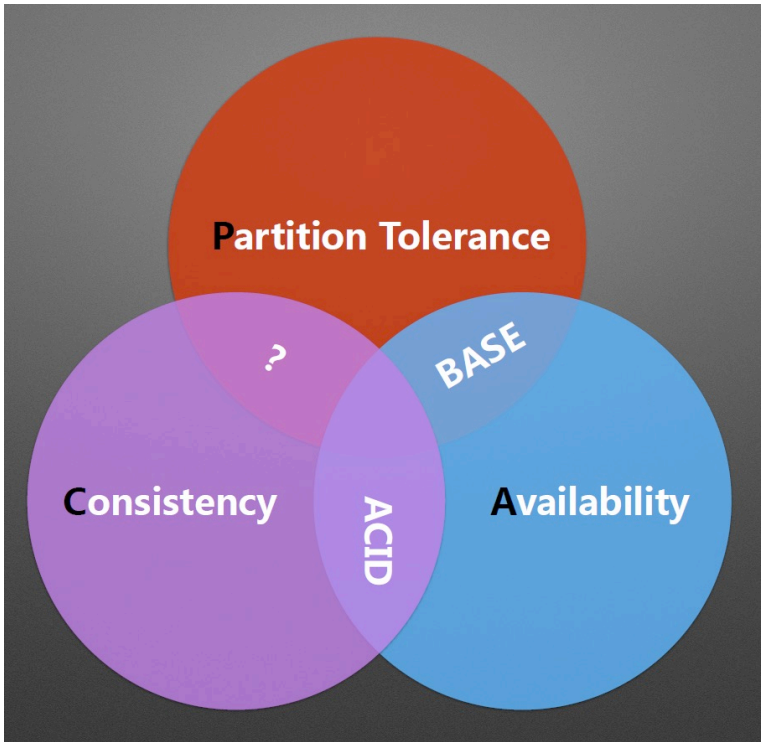
- 一致性

  数据对象在不同副本之间一致；

- 共识

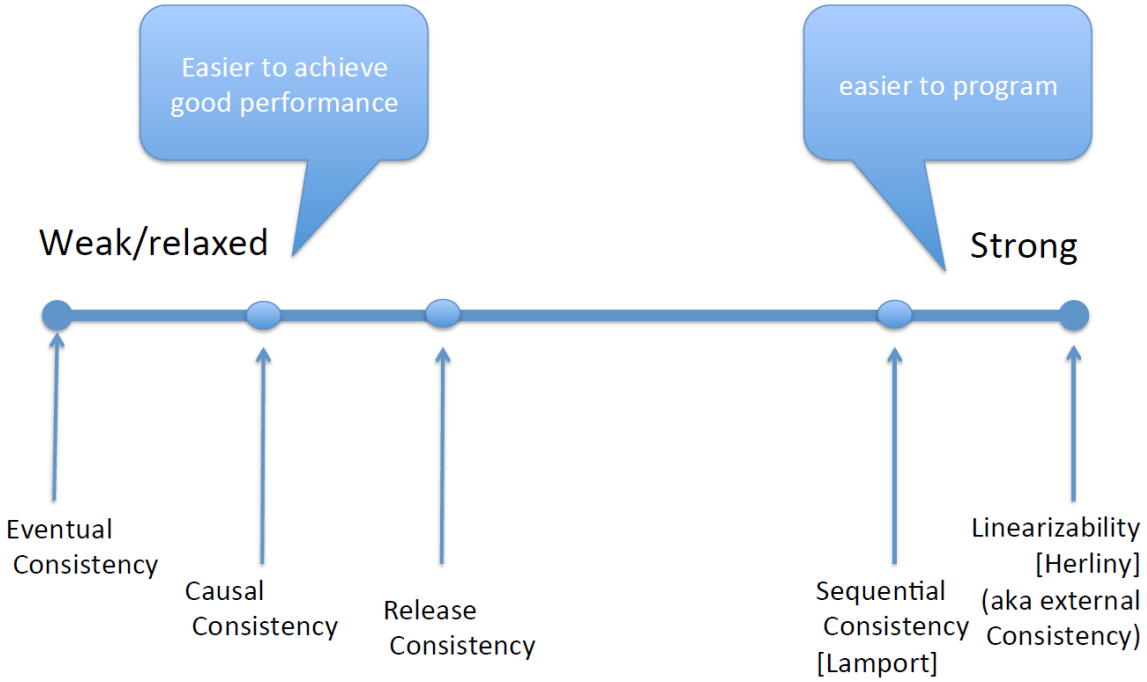
  多个节点对于某件事情比如执行某个指令达成一致；

背景



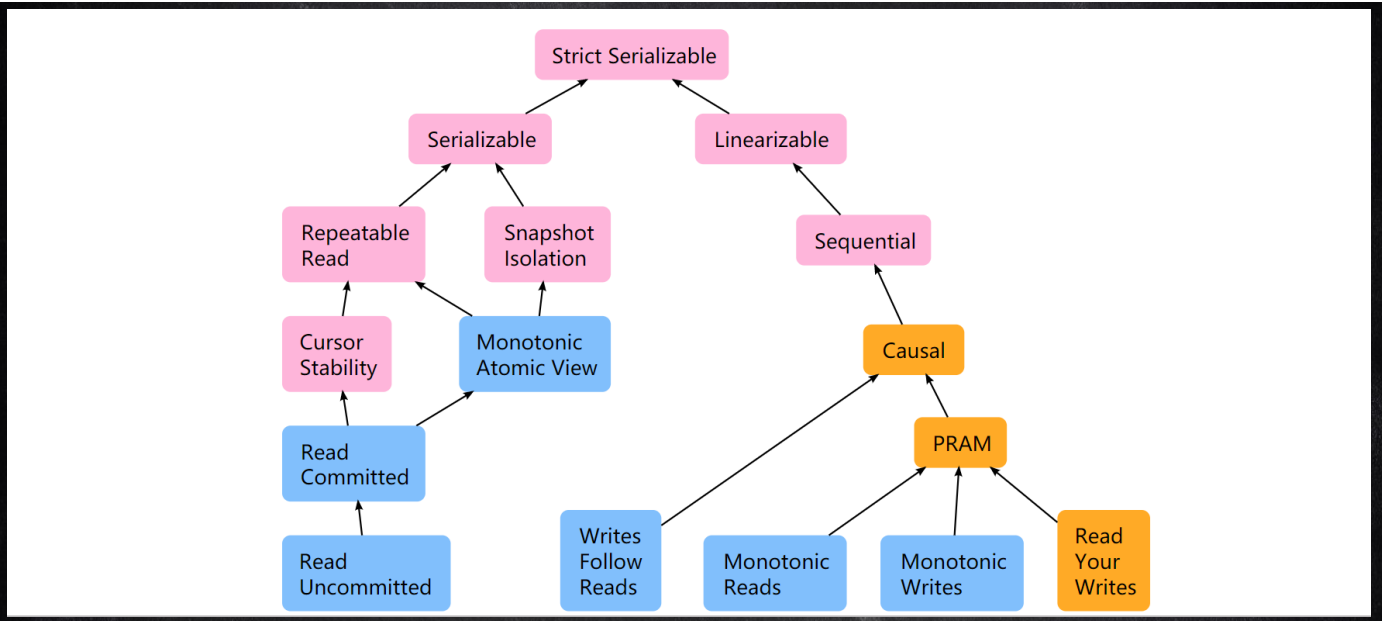
Basically Available、Soft-state、Eventual Consistency（BASE)

Spectrum of Consistency Models









http://jepsen.io/consistency

Unavailable	Not available during some types of network failures. Some or all nodes must pause operations in order to ensure safety.
Sticky Available	Available on every non-faulty node, so long as clients only talk to the same servers, instead of switching to new ones.
Total Available	Available on every non-faulty node, even when the network is completely down.





共识 Paxos







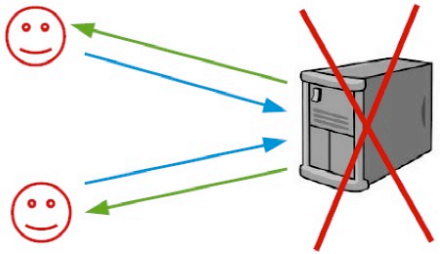
What is the Problem?

Two Generals' Problem

- Two generals need to be agree on time to attack to win.
- They communicate through messengers, who may be killed on their way.
- Agreement is the problem.



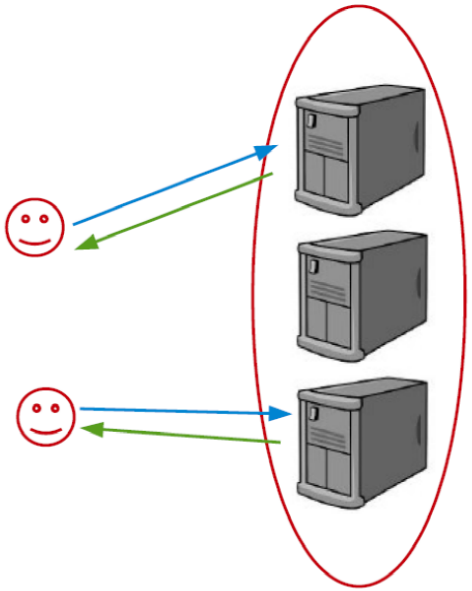
Replicated State Machine Problem (1/2)



- The solution: replicate the server.

Replicated State Machine Problem (2/2)

- Make the server deterministic (state machine).
- Replicate the server.
- Ensure correct replicas step through the same sequence of state transitions (How?)
- Agreement is the problem.









The Agreement Problem





The Agreement Problem

- Some nodes propose values (or actions) by sending them to the others.
- All nodes must decide whether to accept or reject those values.

- But, ...

- Concurrent processes and uncertainty of timing, order of events and inputs.
- Failure and recovery of machines/processors, of communication channels.





Agreement Requirements

- Safety
  - Validity: only a value that has been proposed may be chosen.
  - Agreement: no two correct nodes choose different values.
  - Integrity: a node chooses at most once.
- Liveness
  - Termination: every correct node eventually choose a value.

Agreement in Distributed Systems: Possible Solutions

- Two-Phase Commit (2PC)
- Paxos









Two-Phase Commit (2PC)

The Two-Phase Commit (2PC) Problem

- The problem first was encountered in database systems.
- Suppose a database system is updating some complicated data structures that include parts residing on more than one machine.
- System model:
  - Concurrent processes and uncertainty of timing, order of events and inputs (asynchronous systems).
  - Failure and recovery of machines/processors, of communication channels.

Intuitive Example (1/3)

- You want to organize outing with 3 friends at 6pm Tuesday.
  - Go out only if all friends can make it.



Intuitive Example (2/3)

- What do you do?
  - Call each of them and ask if can do 6pm on Tuesday (voting phase)
  - If all can do Tuesday, call each friend back to ACK (commit)
  - If one cannot do Tuesday, call other three to cancel (abort)







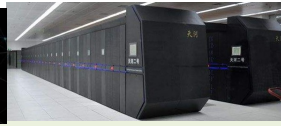
Intuitive Example (3/3)

- Critical details
  - While you were calling everyone to ask, people who have promised they can do 6pm Tuesday must reserve that slot.
  - You need to remember the decision and tell anyone whom you have not been able to reach during commit/abort phase.
- That is exactly how 2PC works.

The 2PC Players

- Coordinator (Transaction Manager)
  - Begins transaction.
  - Responsible for commit/abort.
- Participants (Resource Managers)
  - The servers with the data used in the distributed transaction.





The 2PC Algorithm - Prepare Phase

- Coordinator asks each participant `canCommit`.
- Participants must prepare to commit using permanent storage before answering `yes`.
  - Lock the objects.
  - Participants are not allowed to cause an abort after it replies `yes` to `canCommit`.
- Outcome of the transaction is uncertain until `doCommit` or `doAbort`.
  - Other participants might still cause an abort.

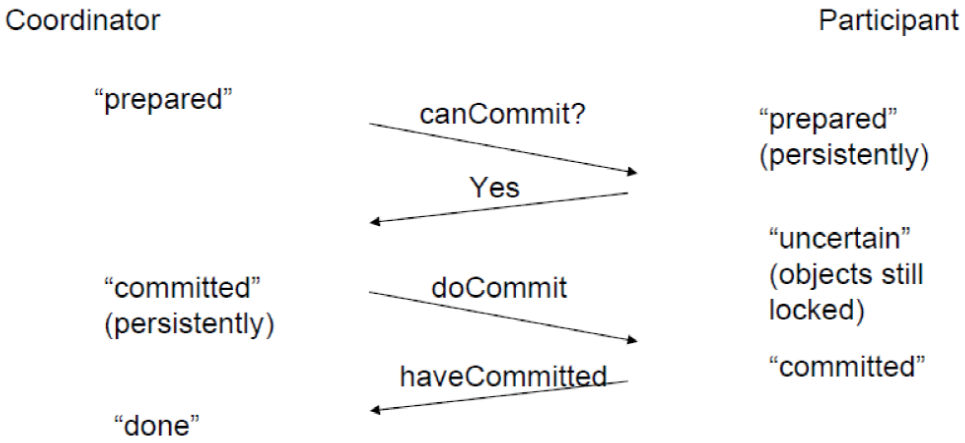




The 2PC Algorithm - Commit Phase

- The coordinator collects all votes.
  - If unanimous yes, causes commit.
  - If any participant voted no, causes abort.
- The fate of the transaction is decided atomically at the coordinator, once all participants vote.
  - Coordinator records fate using permanent storage.
  - Then broadcasts `doCommit` or `doAbort` to participants.

2PC Sequence of Events







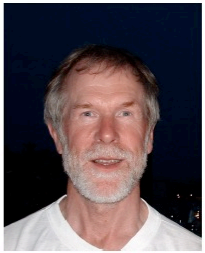


Impossibility of Distributed Consensus with One Faulty Process

FLP

- Fischer-Lynch-Paterson (FLP)
  - M.J. Fischer, N.A. Lynch, and M.S. Paterson, Impossibility of distributed consensus with one faulty process, Journal of the ACM, 1985.





[PDF] Impossibility of Distributed Consensus with One Faulty Process
https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf ▾ 翻译此页
作者：MJ FISCHER - 1985 - 被引用次数：4703 - 相关文章
Impossibility of Distributed Consensus with One Faulty. Process. MICHAEL J. FISCHER. Yale University, New Haven, Connecticut. NANCY A. LYNCH.





FLP Impossibility Result

- It is impossible for a set of processors in an asynchronous system to agree on a binary value, even if only a single process is subject to an unannounced failure.
- The core of the problem is asynchrony.





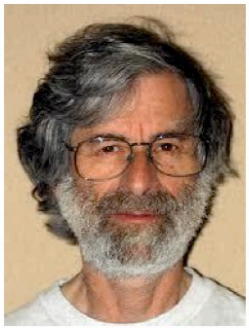
FLP Impossibility Result

- What FLP says: you cannot guarantee both safety and progress when there is even a single fault at an inopportune moment.
- What FLP does not say: in practice, how close can you get to the ideal (always safe and live)?
- So, Paxos ...

Paxos

- The only known completely-safe and largely-live agreement protocol.
- L. Lamport, The part-time parliament, ACM Transactions on Computer Systems, 1998.

希腊岛屿Paxon 上的执法者（legislators，后面称为牧师priest）在议会大厅（chamber）中表决通过法律，并通过服务员传递纸条的方式交流信息，每个执法者会将通过的法律记录在自己的账目（ledger）上。问题在于执法者和服务员都不可靠，他们随时会因为各种事情离开议会大厅，并随时可能有新的执法者进入议会大厅进行法律表决，使用何种方式能够使得这个表决过程正常进行，且通过的法律不发生矛盾。



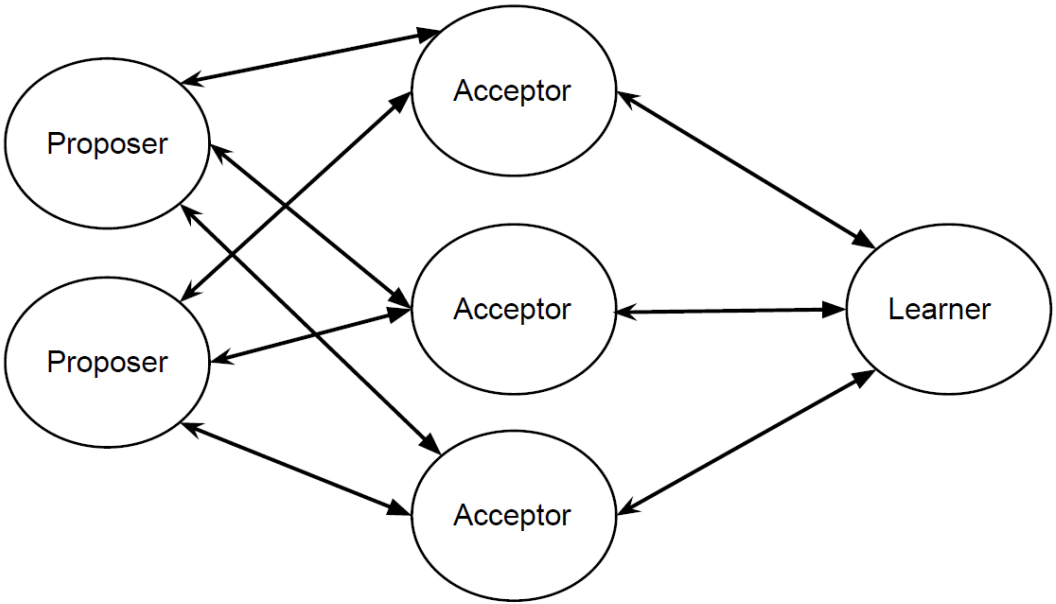




The Paxos Players

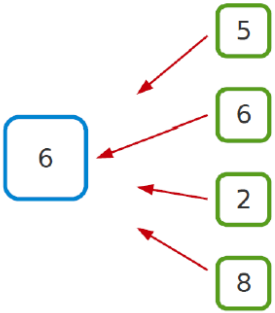
- Proposers
  - Suggests values for consideration by acceptors.
- Acceptors
  - Considers the values proposed by proposers.
  - Renders an accept/reject decision.
- Learners
  - Learns the chosen value.
- A node can act as more than one roles (usually 3).

Paxos Components



Multiple Proposal, Single Acceptor

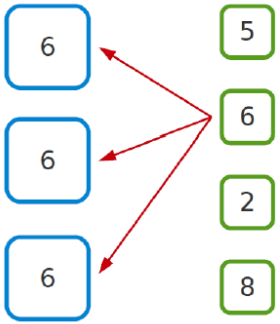
- Use just one acceptor
  - Collects proposers' proposals.
  - Decides the value and tells everyone else.



- Sounds familiar?
  - two-phase commit (2PC)
  - acceptor fails = protocol blocks

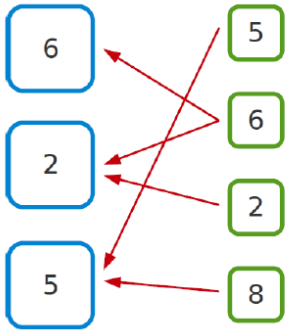
Single Proposal, Multiple Acceptors

- One acceptor is not fault-tolerant enough.
- Let's have multiple acceptors.
- From there, must reach a decision. How?
- Decision = value accepted by the majority.
- P1: an acceptor must accept first proposal it receives.



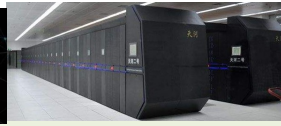
Multiple Proposals, Multiple Acceptors

- If there are multiple proposals, no proposal may get the majority.
  - 3 proposals may each get 1/3 of the acceptors.



- Solution: acceptors can accept multiple proposals, distinguished by a unique proposal number.





Paxos 定义

Proposer: 发起paxos的进程.

Acceptor: 存储节点，接受、处理和存储消息.

Quorum(Acceptor的多数派) : n/2+1个Acceptors.

Round: 1轮包含2个阶段: Phase-1 & Phase-2

每1轮的编号 (**rnd**):

单调递增; 后写胜出; 全局唯一(用于区分Proposer);





Paxos 定义

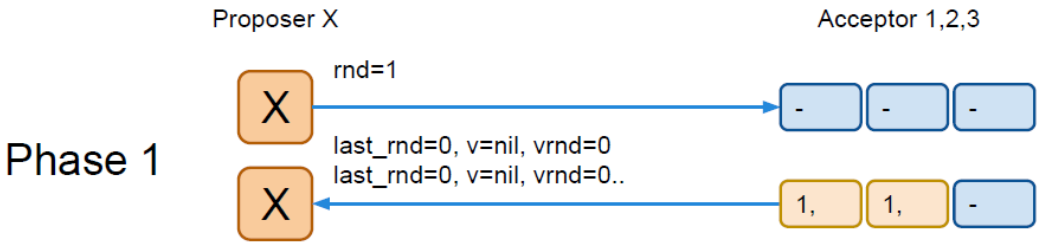
Acceptor看到的最大**rnd** (**last_rnd**):
Acceptor记住这个值来识别哪个proposer可以写。

Value (**v**): Acceptor接受的值.

Value round number (**vrnd**):
Acceptor接受的**v**的时候的**rnd**

值‘被确定的’定义:
有多数(多于半数)个Acceptor接受了这个值.

典型 Paxos：阶段1

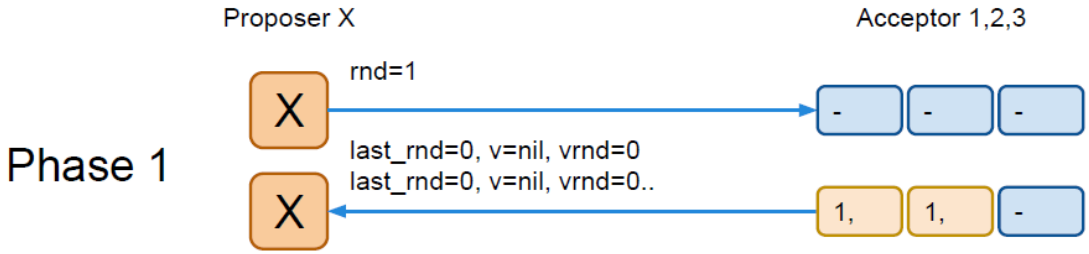


当Acceptor收到phase-1的请求时：

- 如果请求中**rnd**比Acceptor的**last_rnd**小，则拒绝请求
- 将请求中的**rnd**保存到本地的**last_rnd**.

  从此这个Acceptor只接受带有这个**last_rnd**的**phase-2**请求。
- 返回应答，带上自己之前的**last_rnd**和之前已接受的**v**.

典型 Paxos：阶段1

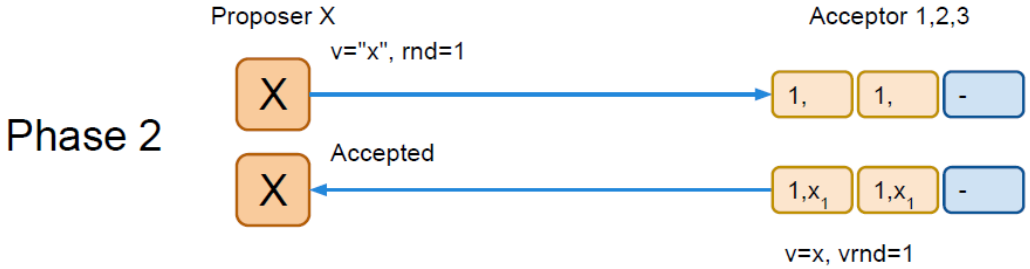


当Proposer收到Acceptor发回的应答：

- 如果应答中的**last_rnd**大于发出的**rnd**: 退出.
- 从所有应答中选择**vrnd**最大的**v**:

  不能改变(可能)已经确定的值
- 如果所有应答的**v**都是空，可以选择自己要写入**v**.
- 如果应答不够多数派，退出

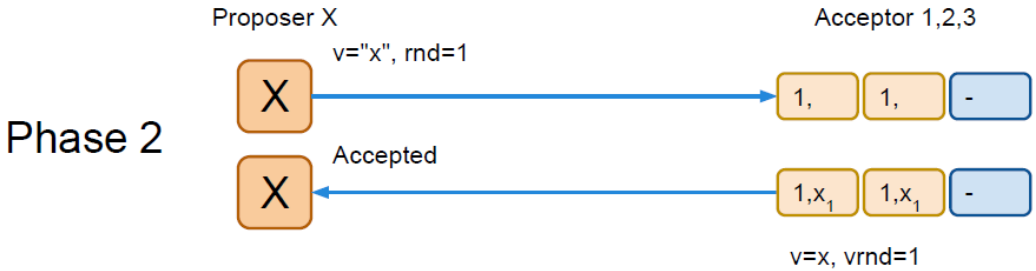
典型 Paxos：阶段2



Proposer:

发送**phase-2**，带上**rnd**和上一步决定的**v**

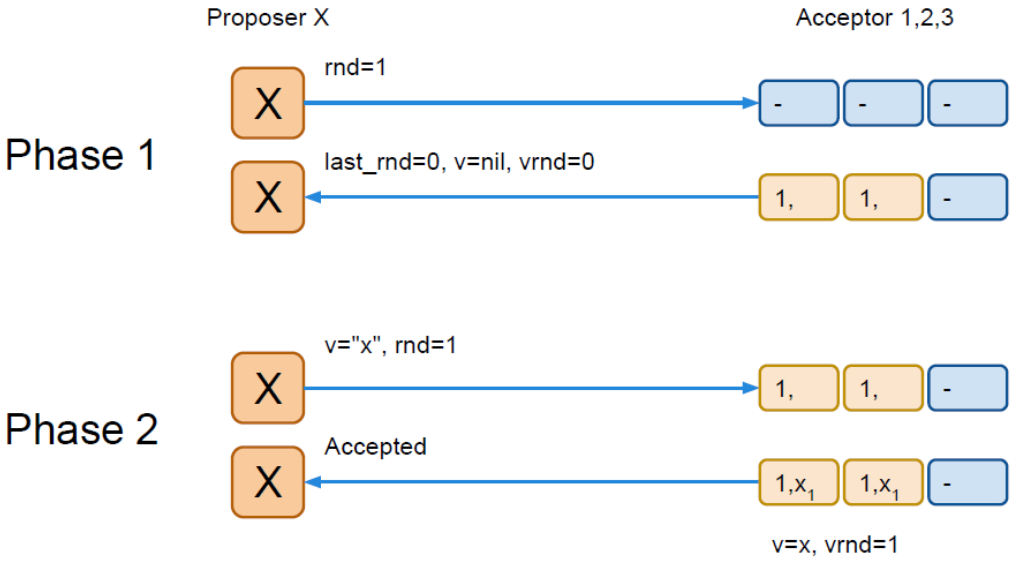
典型 Paxos：阶段2



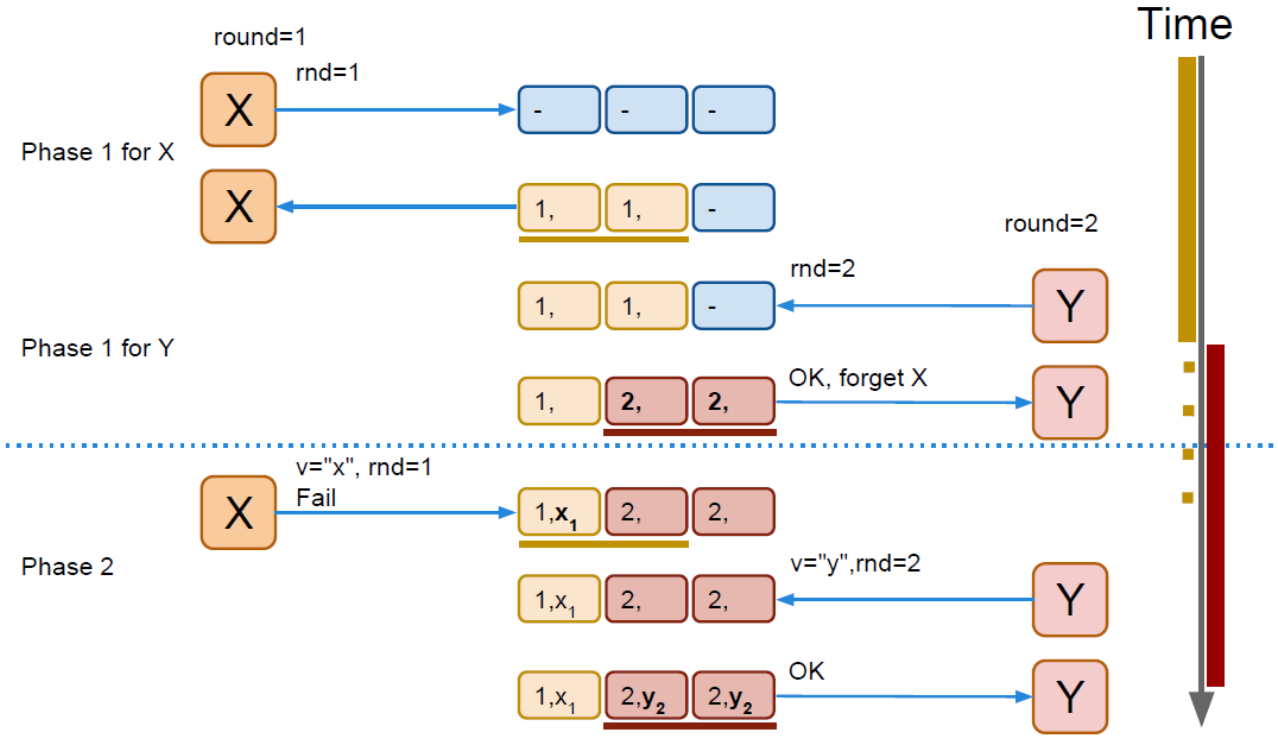
Acceptor:

- 拒绝**rnd**不等于Acceptor的**last_rnd**的请求
- 将**phase-2**请求中的**v**写入本地，记此**v**为‘已接受的值’
- **last_rnd==rnd** 保证没有其他Proposer在此过程中写入过其他值

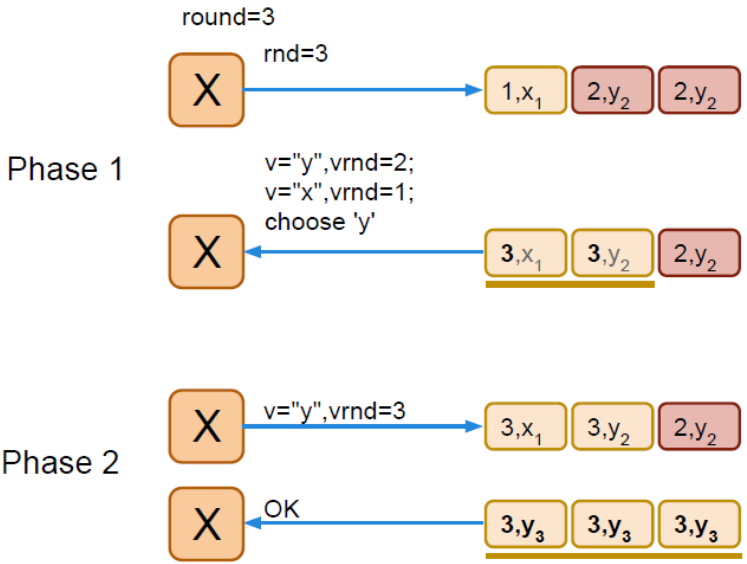
例子： 无冲突


Proposer X
Acceptor 1,2,3
Phase 1
rnd=1
X
-
-
-
last_rnd=0, v=nil, vrnd=0
X
1,
1,
-
Phase 2
v="x", rnd=1
X
1,
1,
-
Accepted
X
1,x1
1,x1
-
v=x, vrnd=1

例子2： 解决并发写冲突


Time
round=1
rnd=1
X
Phase 1 for X
-
-
-
X
1,
1,
-
round=2
rnd=2
1,
1,
-
Y
Phase 1 for Y
1,
2,
2,
OK, forget X
Y
v="x", rnd=1
Fail
X
Phase 2
1,x1
2,
2,
v="y",rnd=2
1,x1
2,
2,
Y
OK
1,x1
2,y2
2,y2
Y

例子3：X 不会修改确定的 V



X只能选择v="y"，因为它可能是一个被确定的值。

其他

Learner角色:

- Acceptor发送**phase-3** 到所有learner角色，让learner知道一个值被确定了.
- 多数场合Proposer就是1个Learner.

Livelock:

多个Proposer并发对1个值运行paxos的时候，可能会互相覆盖对方的**rnd**，然后提升自己的rnd再次尝试，然后再次产生冲突，一直无法完成



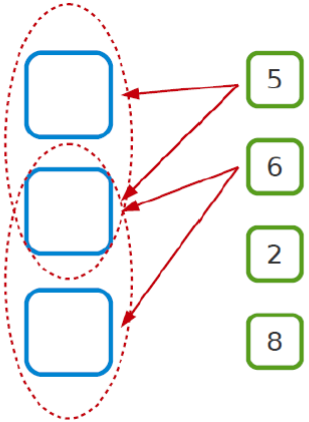


Paxos - Safety (1/3)

- If a value v is chosen at proposal number n, any value that is sent out in phase 2 of any later proposal numbers must be also v.

Paxos - Safety (2/3)

- Decision = Majority (any two majorities share at least one element)
- Therefore after the first round in which there is a decision, any subsequent round involves at least one acceptor that has accepted v.







Paxos - Safety (3/3)

- Now suppose our claim is not true, and let m is the first proposal number that is later than n and in 2nd phase, the value sent out is $w \neq v$.
- This is not possible, because if the proposer P was able to start 2nd phase for w, it means it got a majority to accept round for m (for $m > n$). So, either:
  - v would not have been the value decided, or
  - v would have been proposed by P
- Therefore, once a majority accepts v, that never changes.





Liveness

- If two or more proposers race to propose new values, they might step on each other toes all the time.
  - P_1: `prepare`(n_1)
  - P_2: `prepare`(n_2)
  - P_1: `accept`(n_1, v_1)
  - P_2: `accept`(n_2, v_2)
  - P_1: `prepare`(n_3)
  - P_2: `prepare`(n_4)
  - ...
  - $n_1 < n_2 < n_3 < n_4 < \cdots$
- With randomness, this occurs exceedingly rarely.





Paxos：种类

Classic Paxos

1个实例(确定1个值)写入需要2轮RPC.

Multi Paxos

约为1轮RPC，确定1个值(第1次RPC做了合并).

Fast Paxos

没冲突：1轮RPC确定一个值.

有冲突：2轮RPC确定一个值.



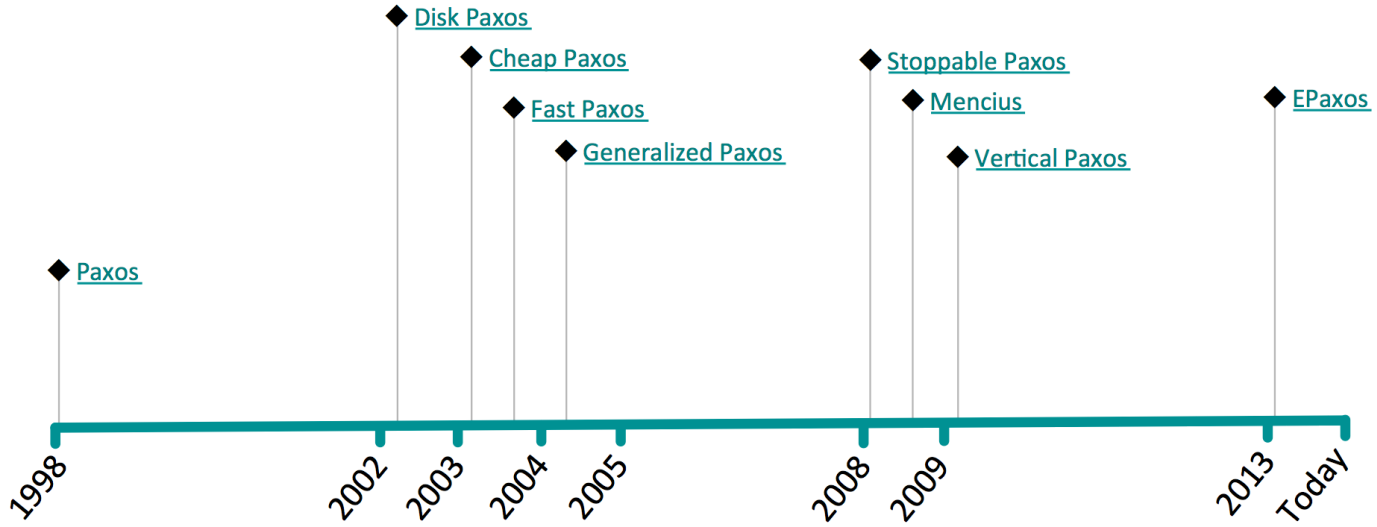


Multi Paxos

将多个paxos实例的phase-1合并到1个RPC；
使得这些paxos只需要运行phase-2即可。

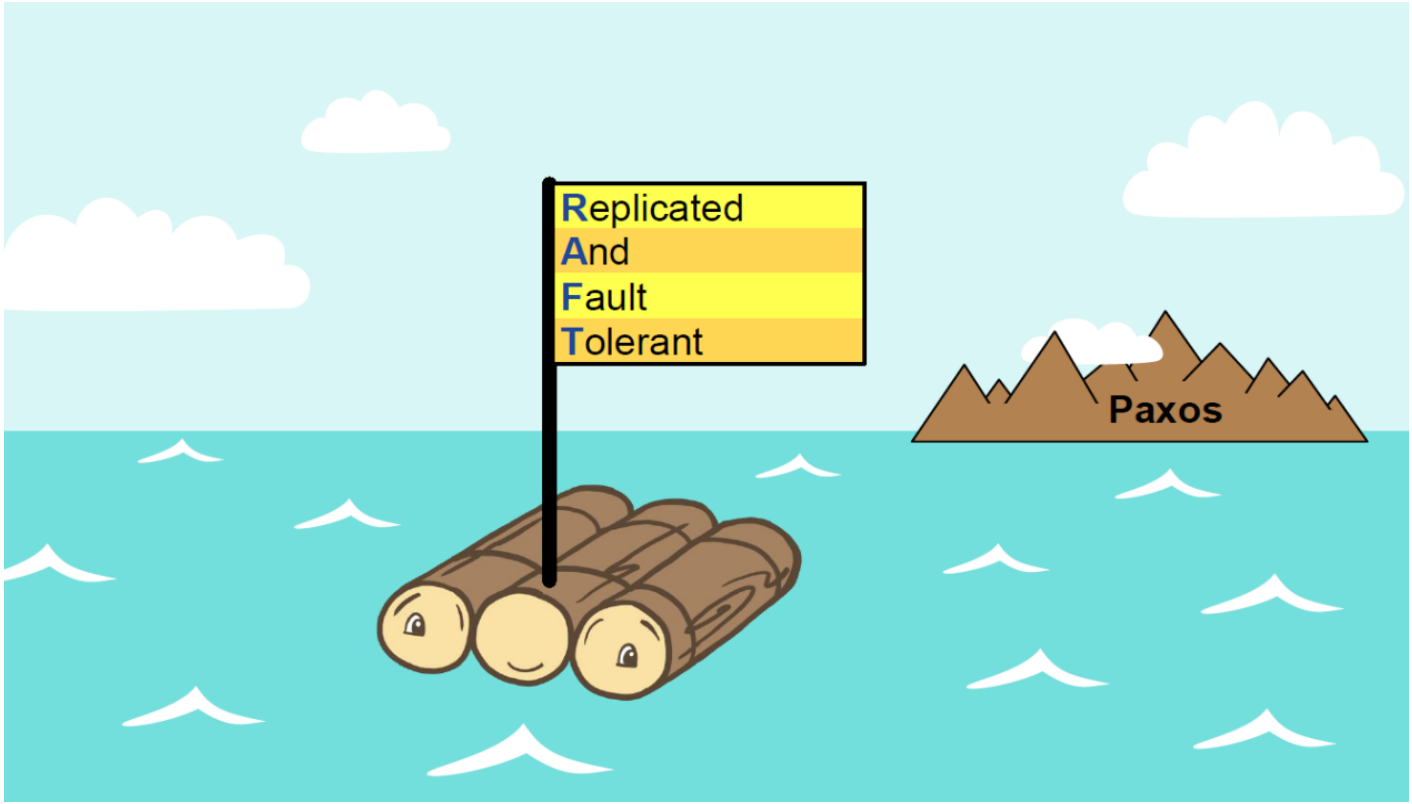
应用:
chubby zookeeper megastore spanner

其他变种



http://paxos.systems/variants.html

RAFT(Replicated And Fault Tolerant)







Algorithms Should Be Designed For …

Correctness?

Efficiency?

Conciseness?

简洁

Understandability!



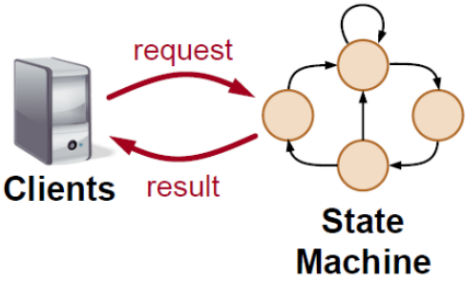


Overview

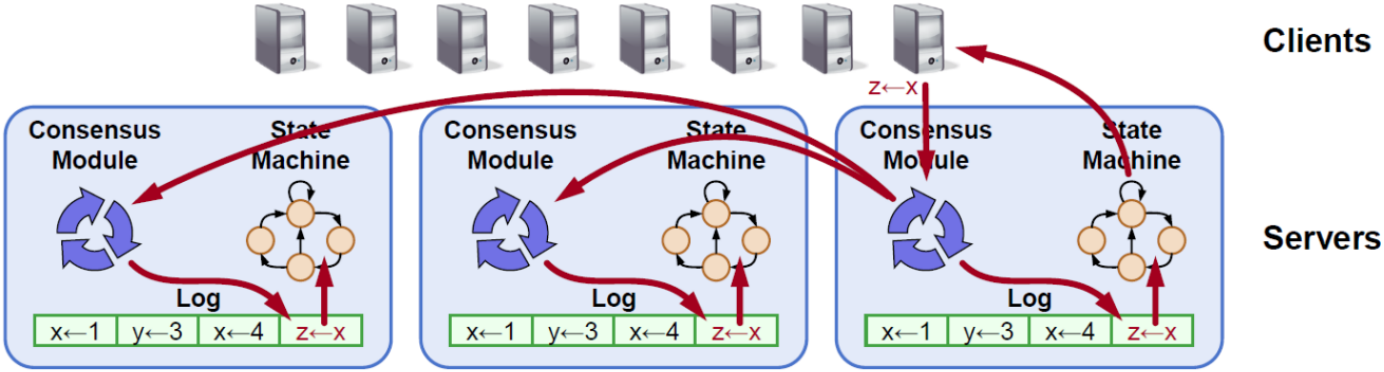
- **Consensus:**
  - Allows collection of machines to work as coherent group
  - Continuous service, even if some machines fail
- **Paxos has dominated discussion for 25 years**
  - Hard to understand
  - Not complete enough for real implementations
- **New consensus algorithm: Raft**
  - Primary design goal: understandability (intuition, ease of explanation)
  - Complete foundation for implementation
  - Different problem decomposition
- **Results:**
  - User study shows Raft more understandable than Paxos
  - Widespread adoption

State Machine

- **Responds to external stimuli**
- **Manages internal state**
- **Examples: many storage systems, services**
  - Memcached
  - RAMCloud
  - HDFS name node
  - ...



Replicated State Machine

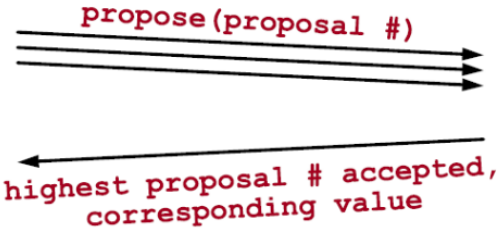


- Replicated log ensures state machines execute same commands in same order
- Consensus module ensures proper log replication
- System makes progress as long as any majority of servers are up
- Failure model: delayed/lost messages, fail-stop (not Byzantine)

Paxos (Single Decree)

Proposers

Choose unique proposal #



Majority? Select value for highest proposal # returned; if none, choose own value

Majority? Value chosen

Acceptors

proposal # > any previous?

proposal # >= any previous?





Paxos Problems

- **Impenetrable: hard to develop intuitions**
  - Why does it work?
  - What is the purpose of each phase?
- **Incomplete**
  - Only agrees on single value
  - Doesn't address liveness
  - Choosing proposal values?
  - Cluster membership management?
- **Inefficient**
  - Two rounds of messages to choose one value
- **No agreement on the details**

> *"The dirty little secret of the NSDI community is that at most five people really, truly understand every part of Paxos :-)"*
> *— NSDI reviewer*

> *"There are significant gaps between the description of the Paxos algorithm and the needs of a real-world system ... the final system will be based on an unproven protocol"*
> *— Chubby authors*

Not a good foundation for practical implementations





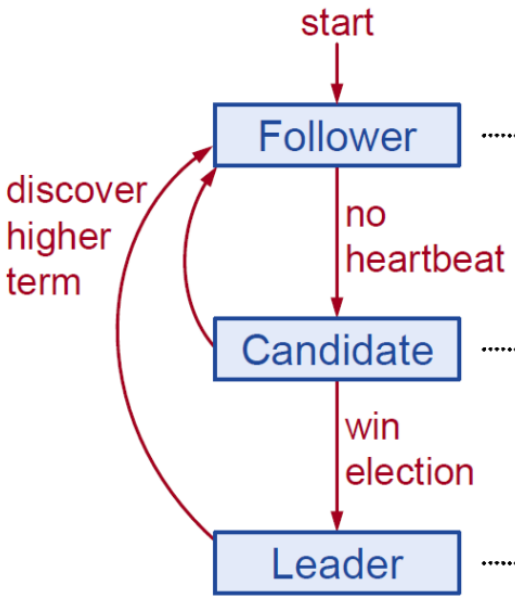
Raft Challenge

- **Is there a different consensus algorithm that's easier to understand?**
- **Make design decisions based on understandability:**
  - Which approach is easier to explain?
- **Techniques:**
  - Problem decomposition
  - Minimize state space
    - Handle multiple problems with a single mechanism
    - Eliminate special cases
    - Maximize coherence
    - Minimize nondeterminism

Raft Decomposition

1. **Leader election:**
   - Select one server to act as leader
   - Detect crashes, choose new leader
2. **Log replication (normal operation)**
   - Leader accepts commands from clients, appends to its log
   - Leader replicates its log to other servers (overwrites inconsistencies)
3. **Safety**
   - Keep logs consistent
   - Only servers with up-to-date logs can become leader

Server States and RPCs



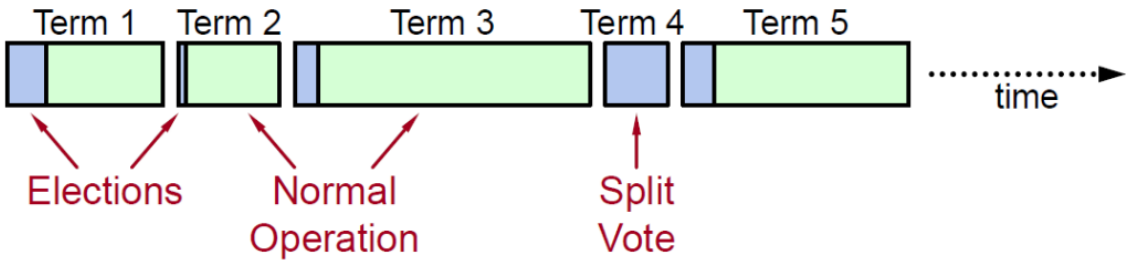
Passive (but expects regular heartbeats)

Issues **RequestVote** RPCs to get elected as leader

Issues **AppendEntries** RPCs:

- Replicate its log
- Heartbeats to maintain leadership

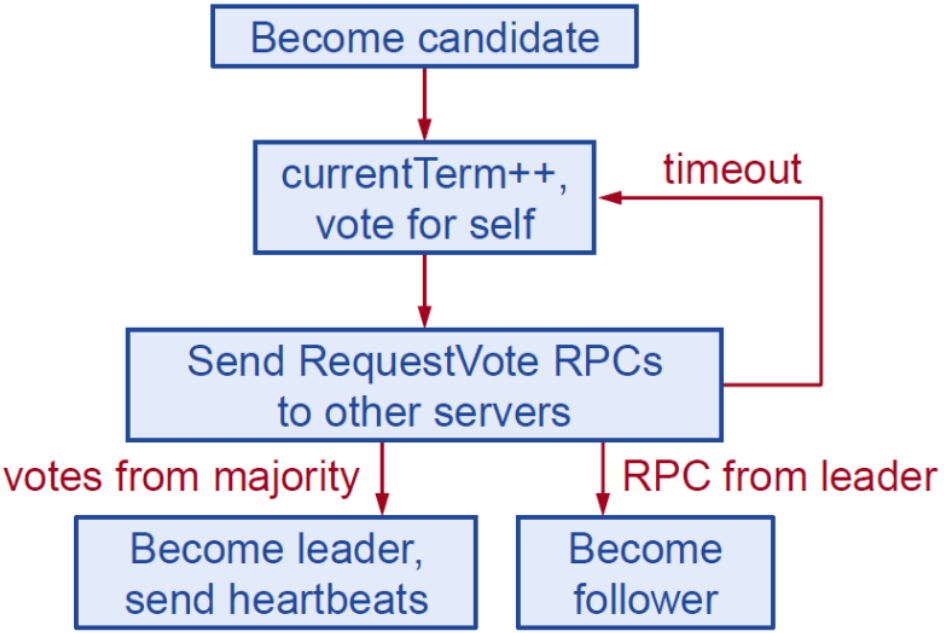
Terms



- **At most 1 leader per term**
- **Some terms have no leader (failed election)**
- **Each server maintains current term value (no global view)**
  - Exchanged in every RPC
  - Peer has later term? Update term, revert to follower
  - Incoming RPC has obsolete term? Reply with error

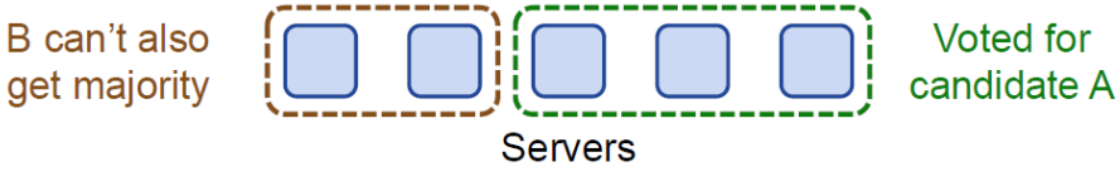
Terms identify obsolete information

Leader Election



Election Correctness

- **Safety: allow at most one winner per term**
  - Each server gives only one vote per term (persist on disk)
  - Majority required to win election



- **Liveness: some candidate must eventually win**
  - Choose election timeouts randomly in [T, 2T] (e.g. 150-300 ms)
  - One server usually times out and wins election before others time out
  - Works well if T >> broadcast time
- **Randomized approach simpler than ranking**

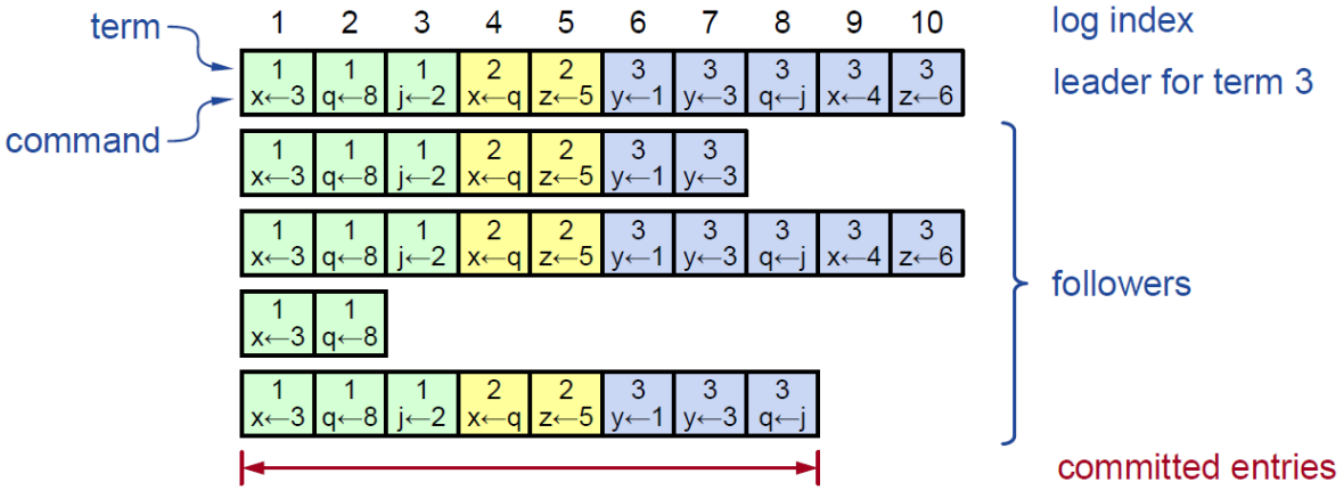




Normal Operation

- **Client sends command to leader**
- **Leader appends command to its log**
- **Leader sends AppendEntries RPCs to all followers**
- **Once new entry committed:**
  - Leader executes command in its state machine, returns result to client
  - Leader notifies followers of committed entries in subsequent AppendEntries RPCs
  - Followers execute committed commands in their state machines
- **Crashed/slow followers?**
  - Leader retries AppendEntries RPCs until they succeed
- **Optimal performance in common case:**
  - One successful RPC to any majority of servers

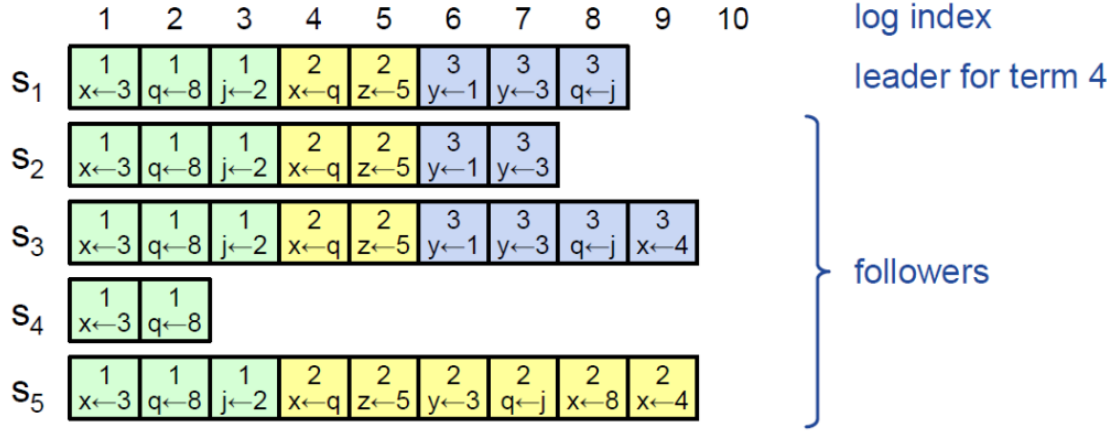
Log Structure



- **Must survive crashes (store on disk)**
- **Entry committed if safe to execute in state machines**
  - Replicated on majority of servers by leader of its term

Log Inconsistencies

Crashes can result in log inconsistencies:



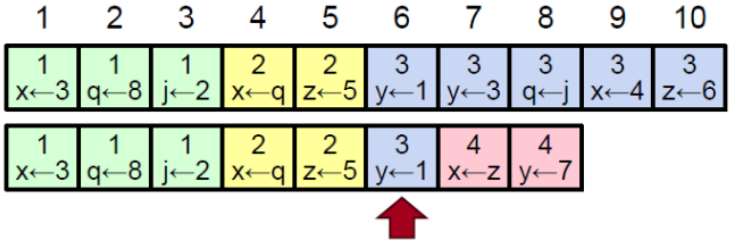
Raft minimizes special code for repairing inconsistencies:

- Leader assumes its log is correct
- Normal operation will repair all inconsistencies

Log Matching Property

Goal: high level of consistency between logs

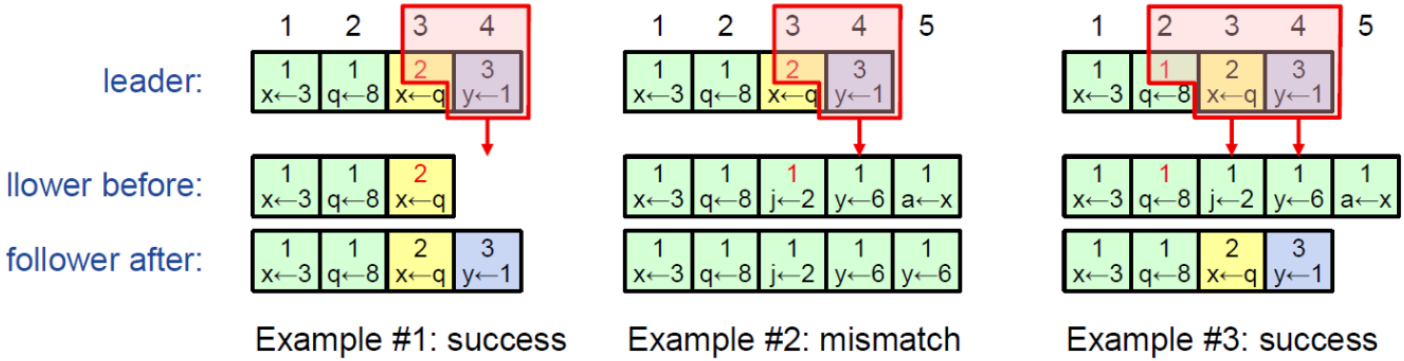
- **If log entries on different servers have same index and term:**
  - They store the same command
  - The logs are identical in all preceding entries



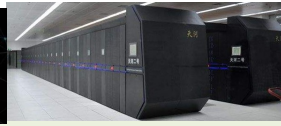
- **If a given entry is committed, all preceding entries are also committed**

AppendEntries Consistency Check

- **AppendEntries RPCs include <index, term> of entry preceding new one(s)**
- **Follower must contain matching entry; otherwise it rejects request**
  - Leader retries with lower log index
- **Implements an induction step, ensures Log Matching Property**







PBFT

Byzantine fault

- A process behaves in an inconsistent manner.
- Can't be solved unless
- n > 3 * f + 1
  - n: number of processes
  - f: number of faults.



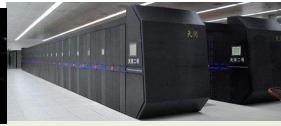


PBFT

Previous Work

- Previous work is not really practical. Too many messages.
- Assume synchrony.
  - Bounds on message delays and process speeds.





PBFT

The Model

- Networks are unreliable
  - Can delay, reorder, drop,retransmit
- Adversary can
  - Can co-ordinate faulty nodes.
  - Delay communication.
- Adversary can't
  - Delay correct nodes.
  - Break cryptographic protocols.
- Some fraction of nodes are byzantine
  - May behave in any way, and need not follow the protocol.
- Nodes can verify the authenticity of messages sent to them.





PBFT

Assumptions about Nodes

- Maintain a state
  - Log
  - View number
  - State
- Can perform a set of operations
  - Need not be simple read/write
  - Must be deterministic
- Well behaved nodes must:
  - start at the same state
  - Execute requests in the same order





PBFT

Views

- Operations occur within views
- For a given view, a particular node in is designated the *primary* node, and the others are *backup* nodes
- Primary = v mod n
  - N is number of nodes
  - V is the view number





PBFT

High level Algorithm

1. A client sends a request to invoke a service operation to the primary
2. The primary multicasts the request to the backups
3. Replicas execute the request and send a reply to the client
4. The client waits for f+1 replies from different replicas with the same result; this is the result of the operation



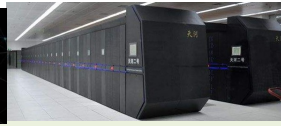


PBFT

High level Algo contd..

- If the client does not receive replies soon enough
  - it broadcasts the request to all replicas. If the request has already been processed, the replicas simply re-send the reply.
- If the replica is not the primary,
  - it relays the request to the primary.
- If the primary does not multicast the request to the group
  - it will eventually be suspected to be faulty by enough replicas to cause a view change



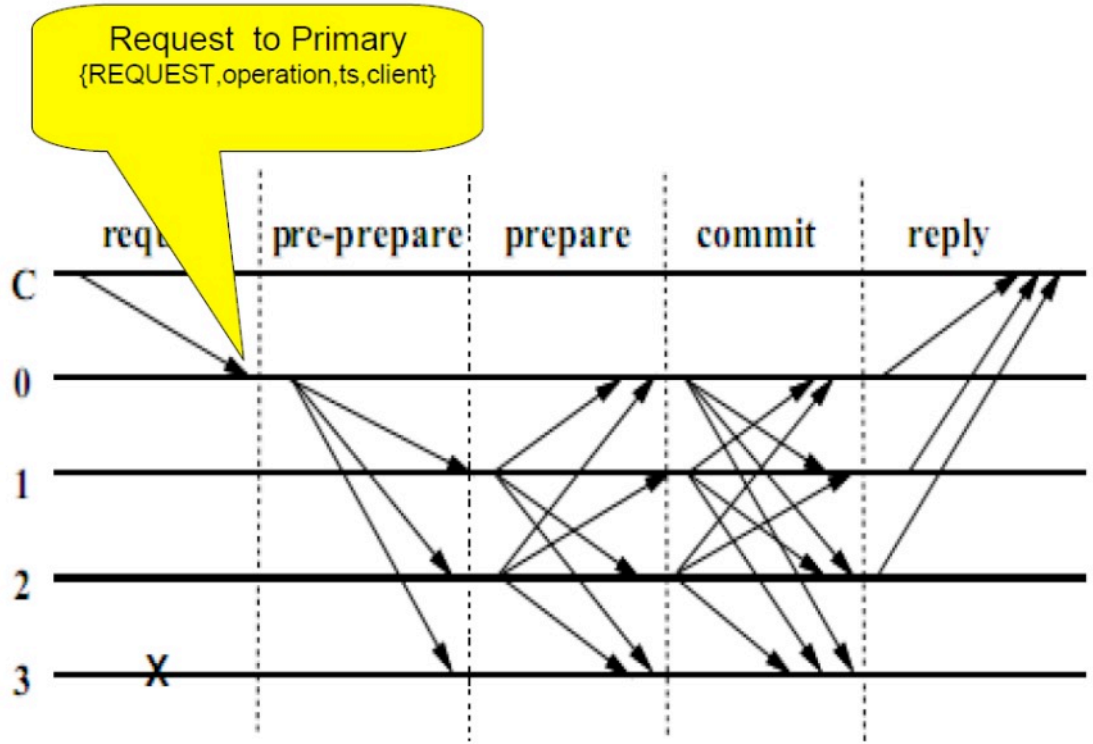


PBFT

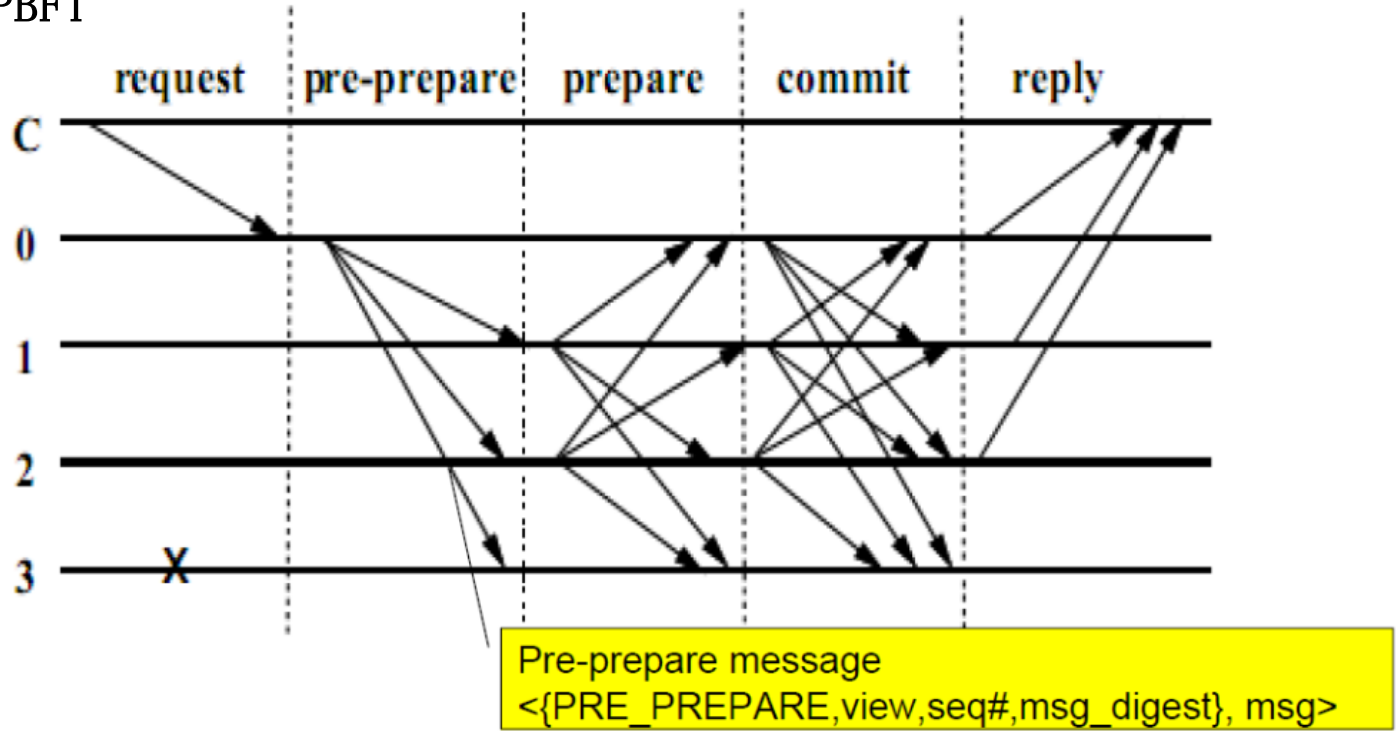
Protocol

- It is a three phase protocol
  - Pre-prepare: primary proposes an order
  - Prepare: Backup copies agree on #
  - Commit: agree to commit

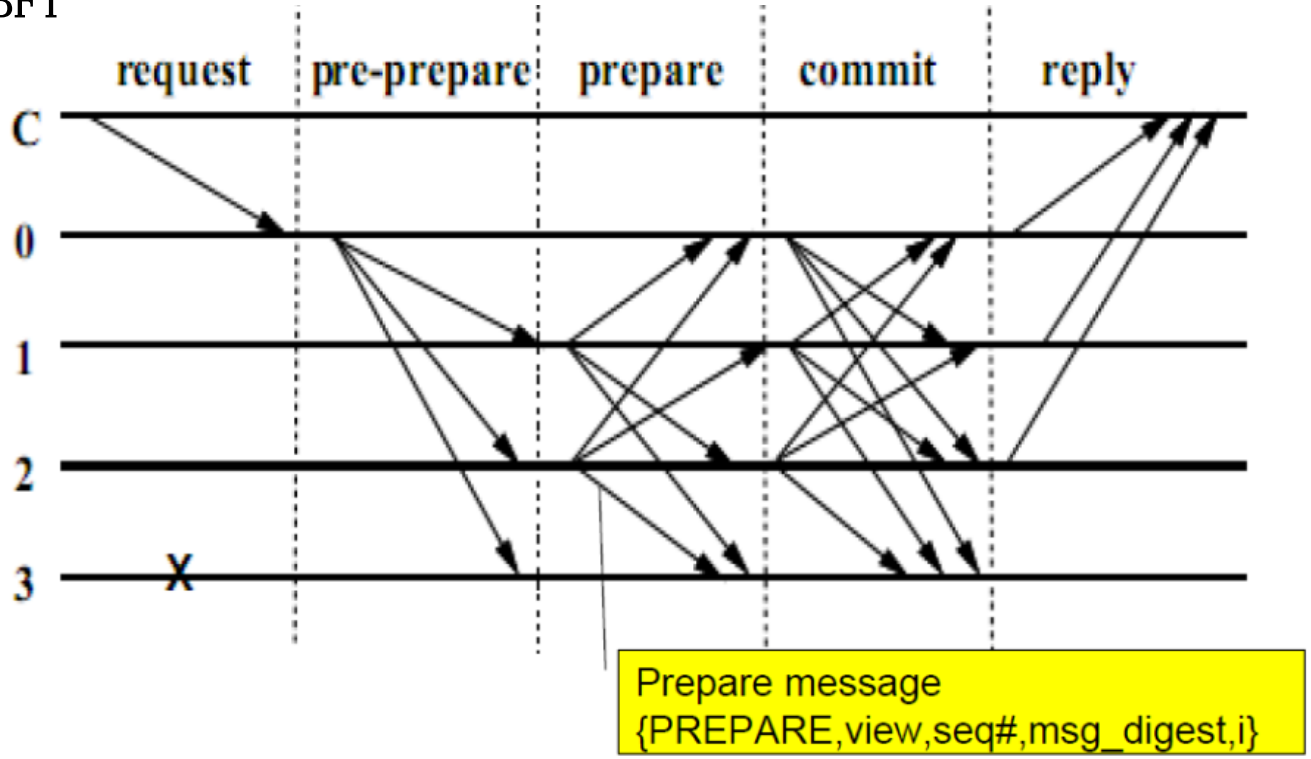
PBFT


Request to Primary
{REQUEST,operation,ts,client}
req
pre-prepare
prepare
commit
reply
C
0
1
2
3
X

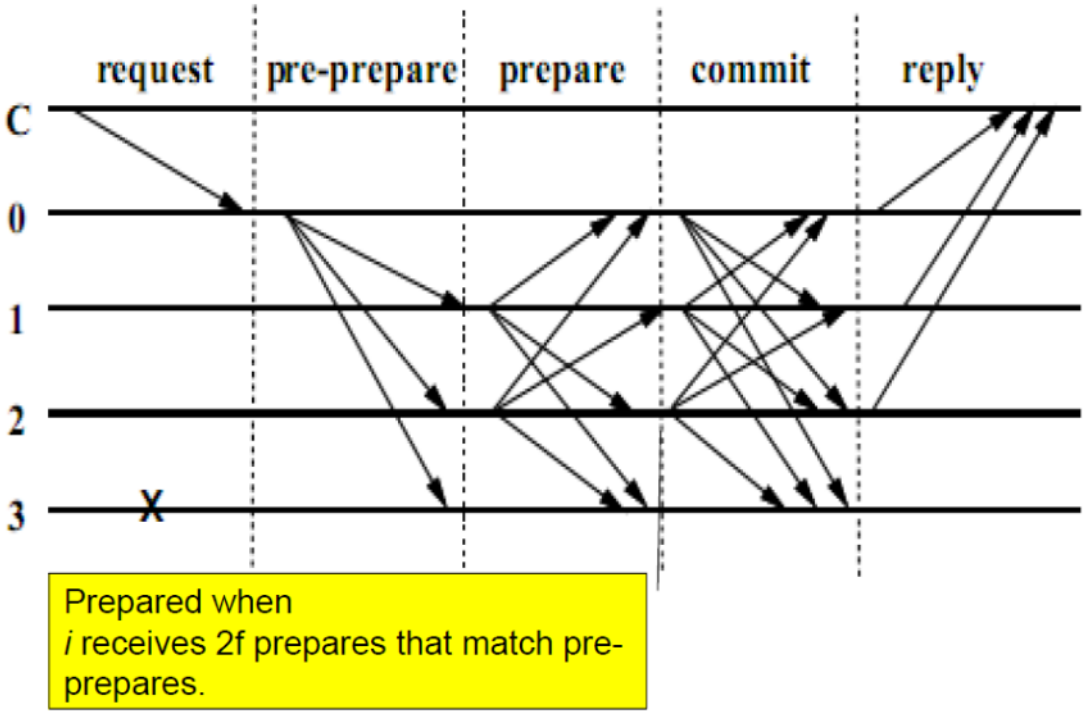
PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Pre-prepare message
<{PRE_PREPARE,view,seq#,msg_digest}, msg>

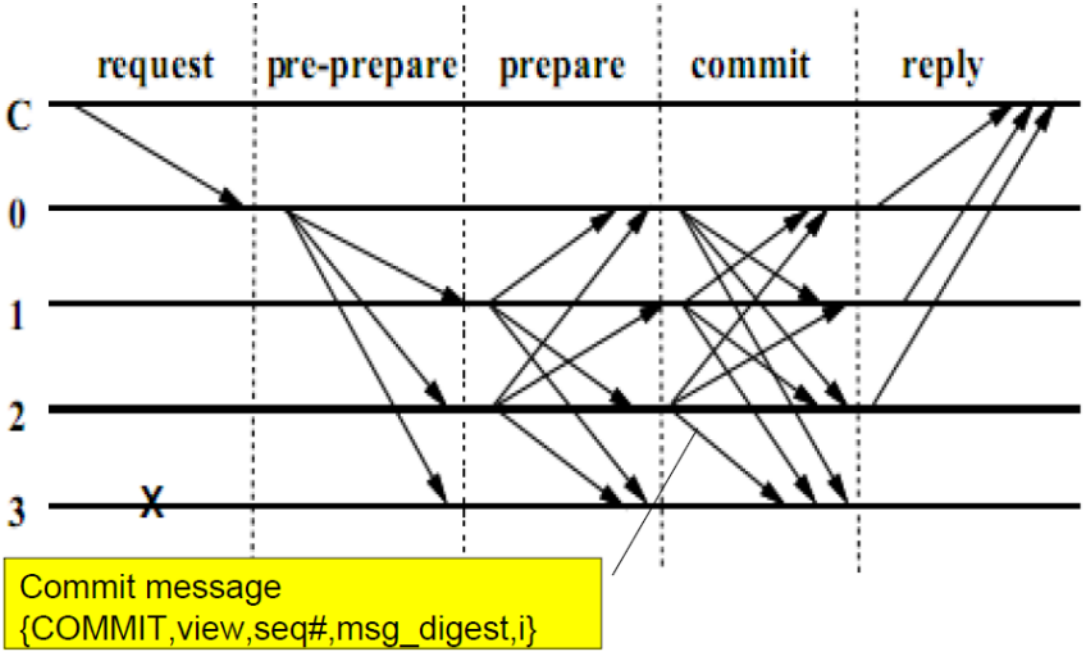
PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Prepare message
{PREPARE,view,seq#,msg_digest,i}

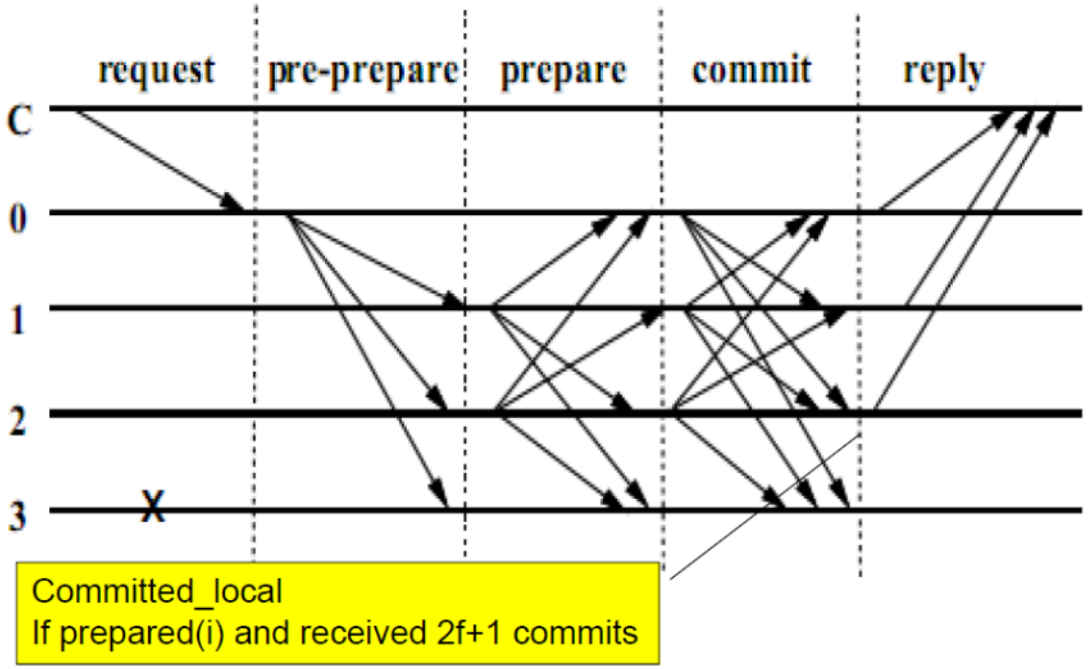
PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Prepared when
i receives 2f prepares that match pre-prepares.

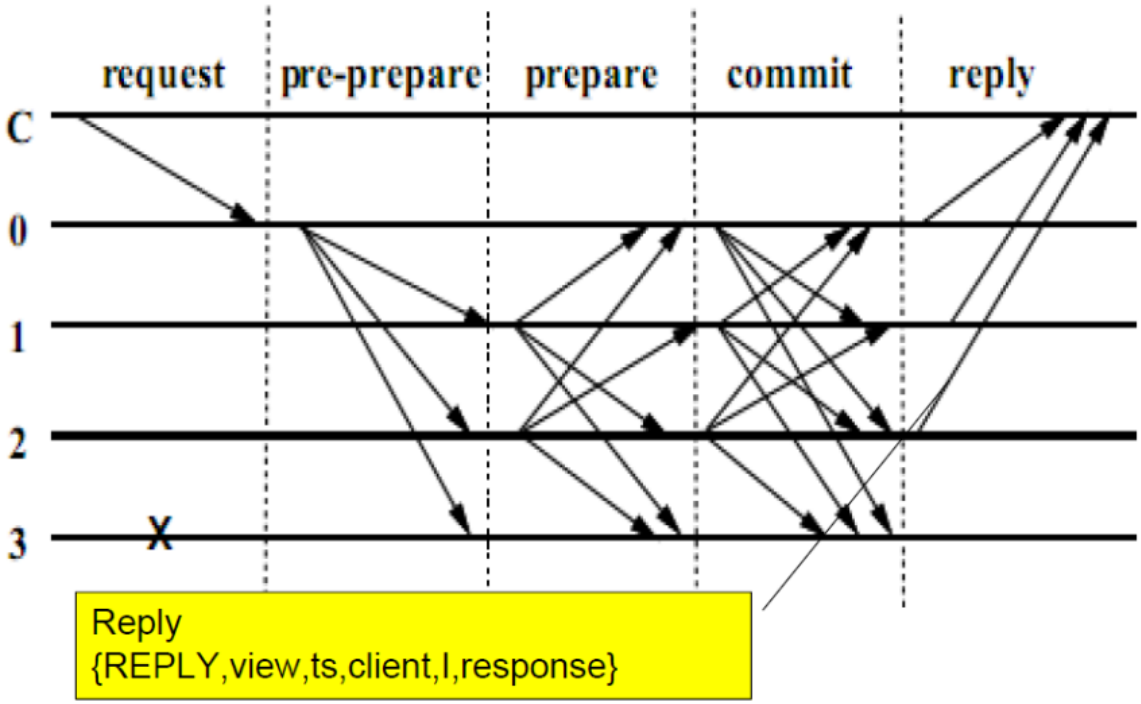
PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Commit message
{COMMIT,view,seq#,msg_digest,i}

PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Committed_local
If prepared(i) and received 2f+1 commits

PBFT


request
pre-prepare
prepare
commit
reply
C
0
1
2
3
X
Reply
{REPLY,view,ts,client,I,response}







谢谢！